

CodeGenie: A Multi-Modal, Offline Chatbot

Author: Manus AI **Date:** December 11, 2025

Abstract

The CodeGenie project introduces a powerful, multi-modal conversational agent designed to operate primarily offline, thereby addressing critical concerns regarding data privacy and dependency on external cloud services. Built upon a Streamlit frontend and a local Large Language Model (LLM) powered by Ollama, CodeGenie seamlessly integrates text-based chat, document processing (PDF, DOCX), Optical Character Recognition (OCR) for image analysis, and voice interaction. This report details the system's architecture, core functionalities, and the workflow for each multi-modal feature.

1. Introduction

1.1. A Versatile, Offline Conversational Agent

CodeGenie is a robust, multi-modal application engineered to process diverse forms of input and deliver intelligent, context-aware responses. Its core strength lies in its ability to function as a self-contained, local AI assistant. The application's primary capabilities include text-based chat, comprehensive document processing for formats such as PDF and DOCX, image analysis via OCR, and intuitive voice interaction.

A key innovation of CodeGenie is its utilization of a locally hosted Large Language Model (LLM) through [Ollama](#), which ensures that core operations and functionality are maintained even without an active internet connection. The entire system is built as a dynamic web application using [Streamlit](#) for the user interface, providing a modern and interactive experience.

2. Problem Statement and Solution

2.1. Reducing Reliance on External Services and Ensuring Privacy

The current landscape of AI tools is characterized by an **over-reliance on external, cloud-based AI services** for even simple tasks. This dependency introduces significant challenges, including potential privacy concerns due to data transmission to third-party servers and a reliance on constant internet connectivity.

The Solution (CodeGenie) is a self-contained, local “self-bot” that fundamentally minimizes the need to engage with other external bots or services. This approach offers several compelling advantages:

- **Offline Capability:** Core chat and processing functions operate entirely without internet access, thanks to the local Ollama LLM.
- **Data Privacy:** User data and conversations remain strictly local, significantly enhancing security and privacy.
- **Multi-Modal Input:** Text, file, and voice inputs are seamlessly integrated into a single, unified conversational flow.

3. System Architecture: Frontend and Backend

3.1. Streamlit and Local LLM Integration

The CodeGenie system is architected around a clear separation of concerns between the user-facing interface and the core processing logic.

Component	Description and Key Libraries
Frontend (Streamlit)	
User Interface	Provides the interactive, web-based user interface.
Session State	Manages chat history (<code>st.session_state.messages</code>), file uploads, and microphone status.
Input Handling	Processes user input and displays chat messages.
Backend/Core Logic	
Local LLM	Ollama (via <code>http://localhost:11434/api/chat</code>) for generating responses.
File Processing	Libraries like <code>pypdf</code> , <code>docx</code> , and <code>PIL</code> for reading various file types.
OCR Engine	<code>easyocr</code> for extracting text from images.
Voice Recognition	<code>speech_recognition</code> for converting spoken words to text.

4. Feature Workflow: Text-Based Search/Chat

The conversational flow is a five-step process that ensures a seamless interaction with the local LLM:

- 1. User Input:** The user types a query into the Streamlit text area.
- 2. Message Append:** The user's message is immediately added to the `st.session_state.messages` list, maintaining the conversation history.
- 3. API Call:** A `requests.post` call is executed, sending the full message history as the payload to the local Ollama API (`http://localhost:11434/api/chat`).
- 4. Response Generation:** The local LLM processes the request based on the provided context and generates a response.
- 5. Display Output:** The assistant's response is received, appended to the message history, and finally rendered in the chat interface for the user.

5. Feature Workflow: Optical Character Recognition (OCR)

The OCR workflow enables the system to extract and utilize text from non-textual sources, integrating them into the LLM's context:

1. **File Upload:** The user uploads an image (PNG, JPG) or a document (PDF, DOCX).
2. **File Type Check:** The `extract_text_from_file` function determines the file's type.
3. **Image Processing (OCR):** For images, the `PIL` library opens the file, and `easyocr.Reader(['en'])` processes the image array to extract text.
4. **Document Processing:** For PDF files, `pypdf.PdfReader` extracts text page by page. For DOCX files, `docx.Document` extracts text paragraph by paragraph.
5. **LLM Context:** The entirety of the extracted text is then added to the chat history as a user message, providing the necessary context for the LLM to process and respond to follow-up queries.

6. Feature Workflow: Speech Searching

The voice-to-text integration allows for hands-free interaction with the chatbot:

1. **Microphone Activation:** The user initiates the process by clicking the microphone button, which sets `st.session_state.mic_active` to `True`.
2. **Listening:** The `capture_voice_input` function utilizes `speech_recognition.Microphone()` to listen for and record audio data.
3. **Voice Recognition:** The recorded audio is sent to Google's Speech Recognition service via `recognizer.recognize_google` for transcription.
4. **Text Conversion:** The spoken words are converted into a text string, with error handling for cases of unknown or unclear speech.
5. **Query Submission:** The resulting transcribed text is used as the `user_input` and submitted to the main chat loop, following the exact same steps as the Text-Based Search workflow (API call to Ollama).

7. Live Demonstration

The live demonstration serves to showcase the seamless integration of CodeGenie's multi-modal capabilities. Key demonstration points include:

- **Offline Chat:** A simple question-and-answer session to highlight the speed and responsiveness of the local LLM.
- **File Upload & OCR:** Uploading an image or PDF and demonstrating how the extracted text is immediately available for the LLM to process and answer questions about.
- **Voice Input:** Using the microphone to ask a question and receiving a text response, illustrating the voice-to-text integration.

Conclusion

CodeGenie successfully integrates multiple input modalities with a powerful, privacy-focused local LLM architecture. By minimizing reliance on external services and ensuring data locality, it provides a robust and secure solution for conversational AI.

Appendix A: Source Code Listing

The following is the primary source code for the CodeGenie application, demonstrating the integration of Streamlit, Ollama API calls, file processing, OCR, and speech recognition.

```
```python
import streamlit as st
import speech_recognition as sr
import requests
from pypdf import PdfReader
from docx import Document
from PIL import Image
import numpy as np
import easyocr
```

## — OLLAMA API URL —

---

url = "<http://localhost:11434/api/chat>"

## — LOAD OCR (EasyOCR only) —

---

```
@st.cache_resource def load_ocr():
```

```
 return easyocr.Reader(['en'])
```

```
ocr_reader = load_ocr()
```

```
from callbacks import (
```

```
 handle_new_chat_click,
 toggle_upload,
 toggle_mic,
 handle_query_submit, # still imported in case you use it
 load_chat
```

```
) from styles import inject_css
```

## — PAGE CONFIG —

---

```
st.set_page_config(
```

```
 page_title="project.py",
 layout="centered"
```

```
)
```

## — SESSION STATE INITIALIZATION —

---

```
if "messages" not in st.session_state:
```

```
st.session_state.messages = []
```

if “chat\_started” not in st.session\_state:

```
st.session_state.chat_started = False
```

if “upload\_clicked” not in st.session\_state:

```
st.session_state.upload_clicked = False
```

if “uploaded\_file” not in st.session\_state:

```
st.session_state.uploaded_file = None
```

if “mic\_active” not in st.session\_state:

```
st.session_state.mic_active = False
```

if “input\_key” not in st.session\_state:

```
st.session_state.input_key = 0
```

if “recent\_chats” not in st.session\_state:

```
st.session_state.recent_chats = []
```

if “listening” not in st.session\_state:

```
st.session_state.listening = False
```

## — VOICE INPUT —

---

def capture\_voice\_input():

```
recognizer = sr.Recognizer()

with sr.Microphone() as source:
 if st.session_state.listening:
 st.info("🔊 Listening... Speak now!")
 audio_data = recognizer.listen(source)
 st.session_state.audio = audio_data

 else:
 if "audio" in st.session_state:
 st.info("⌚ Processing your voice...")

 audio_data = st.session_state.audio
 st.session_state.audio = not st.session_state.audio

 try:
 text = recognizer.recognize_google(audio_data)
 st.success(f"✅ You said: {text}")
 return text

 except sr.UnknownValueError:
 st.warning("❌ Sorry, I couldn't understand your speech.")
 return ""

 except sr.RequestError as e:
 st.error(f"⚠️ Could not request results; {e}")
 return ""
```



# — TEXT EXTRACTION FROM FILES —

---

def extract\_text\_from\_file(uploaded\_file):

```
file_type = uploaded_file.type
extracted_text = ""

try:
 # Images
 if file_type.startswith("image/"):
 image = Image.open(uploaded_file)
 image_np = np.array(image)
 extracted_text = "".join(ocr_reader.readtext(image_np, detail=0))

 # PDF
 elif file_type == "application/pdf":
 reader = PdfReader(uploaded_file)
 for page in reader.pages:
 extracted_text += page.extract_text() or ""

 # DOCX
 elif file_type == "application/vnd.openxmlformats-officedocument.wordprocessingml.document":
 doc = Document(uploaded_file)
 extracted_text = "\n".join(para.text for para in doc.paragraphs)

 # TXT
 elif file_type == "text/plain":
 extracted_text = uploaded_file.read().decode("utf-8")

 else:
 st.warning("Unsupported file type. Please upload image, PDF, or text document.")
 return ""

 return extracted_text.strip()

except Exception as e:
 st.error(f"❌ Error reading file: {e}")
 return ""
```

## — SIDEBAR —

---

def render\_sidebar():

```
with st.sidebar:
 st.button(
 "✚ New Chat",
 on_click=handle_new_chat_click,
 key="new_chat_button",
 use_container_width=True
)

 st.markdown("<h3>💬 Recent chats</h3>", unsafe_allow_html=True)

 for i, chat in enumerate(st.session_state.recent_chats):
 st.button(
 chat["title"],
 on_click=load_chat,
 args=(i,),
 key=f"chat_load_btn_{i}",
 use_container_width=True
)

 st.markdown("""
 <div class="profile">
 <div class="avatar">KA</div>
 <div class="details">
 <div class="name">Karri Manoj</div>
 </div>
 </div>
 """, unsafe_allow_html=True)
```

## — HEADER (Custom Title) —

---

def render\_header():

```
if not st.session_state.chat_started:
 st.markdown("""
 <style>
 [data-testid="stHeader"]::after {
 content: " CodeGenie ";
 font-size: 19px;
 color: #ff4b4b;
 display: flex;
 align-items: center;
 gap: 8px;
 }
 </style>
 """, unsafe_allow_html=True)
```

## — CHAT HISTORY —

---

```
def render_chat_history():
```

```

chat_history_class = "chat-history-container visible" if
st.session_state.chat_started else "chat-history-container"

st.markdown(f'<div class="{chat_history_class}">', unsafe_allow_html=True)

for message in st.session_state.messages:
 role = message["role"]
 content = message["content"]

 row_class = "chat-message-row user-message-row" if role == "user" else
"chat-message-row ai-message-row"
 bubble_class = "chat-message-bubble user-message-bubble" if role ==
"user" else "chat-message-bubble ai-message-bubble"

 st.markdown(
 f"""
 <div class="{row_class}">
 <div class="{bubble_class}">
 {content}
 </div>
 </div>
 """,
 unsafe_allow_html=True
)

st.markdown('</div>', unsafe_allow_html=True)

```

## — INPUT AREA —

---

def render\_input\_area():

```

if st.session_state.chat_started:
 search_bar_class = "search-bar-container fixed-bottom"
else:
 search_bar_class="search-bar-container"

st.markdown(f'<div class="{search_bar_class}">', unsafe_allow_html=True)

col1, col2, col3 = st.columns([0.4, 3.8, 0.4])
current_input_key = f"query_input_value_{st.session_state.input_key}"

with col1:
 st.button("✚", on_click=toggle_upload,
key=f"plus_button_{st.session_state.input_key}")

with col2:
 user_input = st.text_area(
 label="chat_input",
 placeholder="Ask anything....",
 height=2,
 key=current_input_key
)

with col3:
 mic_icon = "🎤" if st.session_state.get("mic_active", False) else "🔇"
 st.button(mic_icon, on_click=toggle_mic,
key=f"mic_button_{st.session_state.input_key}")

Voice input
if st.session_state.get("mic_active", False):
 st.session_state.listening = not st.session_state.listening
 voice_text = capture_voice_input()
 if voice_text:
 user_input = voice_text
 st.session_state.mic_active = False

st.markdown('</div>', unsafe_allow_html=True)

Send message
if user_input:
 st.session_state.chat_started = True
 st.session_state.messages.append({"role": "user", "content":
user_input})

 payload = {
 "model": "llama3.2",

```

```
 "messages": st.session_state.messages,
 "stream": False
 }

 try:
 response = requests.post(url, json=payload).json()
 assistant_msg = response["message"]["content"]
 st.session_state.messages.append({"role": "assistant", "content":
assistant_msg})

 except Exception as e:
 st.error(f"Error contacting Ollama: {e}")
```

## — FILE UPLOADER —

---

```
def render_file_uploader():
```

```

if st.session_state.upload_clicked and st.session_state.uploaded_file is
None:
 st.markdown('<div class="file-uploader-fixed">', unsafe_allow_html=True)
 uploaded_file = st.file_uploader(
 "Upload your file",
 key="file_uploader_main",
 type=["pdf", "png", "jpg", "jpeg", "txt", "docx"],
 label_visibility="collapsed"
)

 if uploaded_file:
 st.session_state.uploaded_file = uploaded_file
 st.toast(f"Uploaded: {uploaded_file.name}")

 extracted_text = extract_text_from_file(uploaded_file)

 if extracted_text:
 st.session_state.chat_started = True
 st.session_state["ocr_text"] = extracted_text

 preview = extracted_text[:500]

 st.session_state.messages.append({
 "role": "user",
 "content": f" Extracted text from
{uploaded_file.name}:\n\n{preview}"
 })

 with st.expander("Show Full Extracted Text"):
 st.text_area("Extracted Text", extracted_text, height=300)

 payload = {
 "model": "llama3.2",
 "messages": st.session_state.messages,
 "temperature": 1,
 "max_tokens_to_sample": 300,
 "stream": False
 }

 try:
 response = requests.post(url, json=payload).json()
 assistant_msg = response["message"]["content"]
 st.session_state.messages.append({"role": "assistant",
"content": assistant_msg})

```

```
except Exception as e:
 st.error(f"Error contacting Ollama: {e}")

st.markdown('</div>', unsafe_allow_html=True)
```

## — MAIN APP —

---

def main():

```
inject_css()
st.markdown(
 '<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.2/css/all.min.css">',
 unsafe_allow_html=True
)

render_header()

if not st.session_state.chat_started:
 st.markdown('<h1 class="title">What\'s on your mind today?</h1>',
 unsafe_allow_html=True)

render_chat_history()
render_file_uploader()
render_input_area()
render_sidebar()
```

if name == "main" :

```
main()
```

...